CPSY 1291 · Recitation handout 4

One neuron, and the calculus of learning

Angles, gratings, derivatives, gradients, and the chain rule

Fall 2026 · Week 4 · hold this after Lecture 8; Assignment 2 is due Sun 10/18

What this session is for

Assignment 2 is about what a single neuron computes and how it learns. Two things it assumes are not taught in lecture, and this is where you get them.

Part A is geometry: the dot product written as $\|\boldsymbol{w}\|\|\boldsymbol{x}\| \cos\theta$, angles in radians, and how to build a sinusoidal grating — the stimulus used to measure orientation tuning in real visual cortex and, in the assignment, in a network.

Part B is calculus: derivatives, partial derivatives, the gradient, the chain rule, and gradient descent. Every learning rule in this course is one line of calculus applied to an error. Once you can differentiate a squared error, the delta rule stops being something to memorize and becomes something you can rederive on the exam in thirty seconds.

If you have had a calculus course, Part B is revision and you can skim it. If you have not, read it slowly — it is written for you, and nothing beyond it is required all term.

> **The one idea.** A derivative answers exactly one question: *if I nudge this input a little, how much does the output move, and in which direction?* Everything else — gradients, backpropagation, the whole training loop — is that question asked about more variables at once.

Part A — The geometry of one neuron

1 The dot product, written two ways

A linear neuron computes $a = \boldsymbol{w} \cdot \boldsymbol{x} + b$. The same quantity can be written

$$\boldsymbol{w} \cdot \boldsymbol{x} \;=\; \sum_i w_i x_i \qquad \text{and} \qquad \boldsymbol{w} \cdot \boldsymbol{x} \;=\; \|\boldsymbol{w}\|\,\|\boldsymbol{x}\|\,\cos\theta.$$

The first tells you how to compute it. The second tells you what it *means*, and it separates the response into two independent causes:

- $\|\boldsymbol{x}\|$ — how strong the input is (its contrast, its overall firing rate);
- $\cos\theta$ — how well its *pattern* matches the neuron's preferred pattern.

> **Why this matters.** A unit can respond strongly either because the stimulus matched its preference or because the stimulus was simply intense. Those are different scientific claims. Normalizing every stimulus to unit length removes the second cause, and then the response is $\|\boldsymbol{w}\| \cos\theta$ — a pure measure of pattern match.

```
xn = x / np.linalg.norm(x) # unit length: intensity removed
cos = (w @ xn) / np.linalg.norm(w) # in [-1, 1]
theta_deg = np.degrees(np.arccos(np.clip(cos, -1, 1)))
```

The `np.clip` is not paranoia. Floating point returns things like 1.0000000000000002, and `arccos` of that is `nan`.

1.1 Which stimulus drives a unit hardest?

Among all inputs of a fixed length, the response $\|\boldsymbol{w}\|\|\boldsymbol{x}\| \cos\theta$ is largest when $\cos\theta = 1$ — that is, when $\boldsymbol{x}$ points in the same direction as $\boldsymbol{w}$. **The optimal stimulus for a linear unit is its own weight vector.** This is why plotting a unit's weights as an image is a meaningful thing to do, and it is the entire basis for calling the first layer of a network "Gabor-like".

2 Radians, degrees, and gratings

2.1 Angles

NumPy's trigonometric functions take **radians**. A full turn is 2π radians, $180° = \pi$ radians.

```
np.deg2rad(45) # 0.7853... use these two functions and never
np.rad2deg(np.pi) # 180.0 convert by hand
```

Orientation, unlike direction, is defined modulo $180°$: a bar tilted at $10°$ and one at $190°$ are the same bar. When you average or compare preferred orientations, work modulo 180, or a unit preferring $175°$ and one preferring $5°$ will look maximally different when they are $10°$ apart.

2.2 A sinusoidal grating in four lines

A grating is a stripe pattern: brightness varying as a sine wave along one direction. It has an orientation θ, a spatial frequency f (stripes per image), and a phase φ (where the stripes sit).

```
def grating(size=11, theta=0.0, freq=2.0, phase=0.0):
   y, x = np.mgrid[0:size, 0:size] / size # coordinates in [0, 1)
   proj = x * np.cos(theta) + y * np.sin(theta) # distance along theta
   return np.sin(2 * np.pi * freq * proj + phase)
```

The middle line is the idea: project each pixel's position onto the direction θ, then make brightness a sine of that projection. Everything perpendicular to θ has the same projection,

which is why the stripes come out parallel.

2.3 Tuning curves, and the number to report

A **tuning curve** is a unit's response plotted against a stimulus parameter. Measure it by sweeping the parameter and recording the response.

Phase is a nuisance parameter: a unit that likes vertical stripes may respond strongly to one phase and weakly to its opposite, purely because the light and dark bars swapped. The standard fix is to average over several phases at each orientation, which is what makes the curve about orientation and not about where the stripes happened to land.

The usual summary of how sharp a tuning curve is:

> **Half-width at half-maximum (HWHM).** Find the peak. Find the two orientations either side where the response has fallen to halfway between the peak and the baseline. Half the distance between them is the bandwidth. Smaller means more sharply tuned. Macaque V1 simple cells sit near $23°$.

Part B — The calculus of learning

3 A derivative is a slope

$f'(x)$ — also written $\frac{df}{dx}$ — is the slope of f at x: how much f changes per unit change in x.

$f(x)$	$f'(x)$	where you meet it
c (a constant)	0	biases that are not being trained
x	1	everywhere
x^2	$2x$	squared error
e^x	e^x	exponentials, softmax
$\log x$	$1/x$	cross-entropy, surprisal
$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)\,(1-\sigma(x))$	sigmoid units
$\tanh x$	$1-\tanh^2 x$	RNNs
$\mathrm{ReLU}(x) = \max(0,x)$	1 if $x>0$, else 0	almost every network

Two remarks on that table. The sigmoid and tanh derivatives are written in terms of the function's *own output*, which is why a network can compute them cheaply during backpropagation — it already has the output. And ReLU has no derivative at exactly 0; every library simply picks 0 and nothing depends on the choice.

> **The two combining rules, and they are all you need.** Sum: $(f+g)' = f' + g'$. Chain: if $y = f(g(x))$ then $\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$.

4 The chain rule as a pipeline

Read the chain rule as bookkeeping about a pipeline rather than as an identity. Suppose

$$z = \boldsymbol{w} \cdot \boldsymbol{x} + b, \qquad a = \mathrm{ReLU}(z), \qquad L = \tfrac{1}{2}(a - y)^2.$$

To ask how the loss responds to w_i, walk backwards and multiply the local slopes:

$$\frac{\partial L}{\partial w_i} = \underbrace{(a - y)}_{\partial L/\partial a} \cdot \underbrace{\mathbb{1}[z > 0]}_{\partial a/\partial z} \cdot \underbrace{x_i}_{\partial z/\partial w_i}.$$

Each factor is local: it involves only one stage and quantities that stage already has. That locality *is* backpropagation — the algorithm is nothing more than this multiplication, done once per layer, from the loss backwards.

> **Where dead ReLUs come from.** If $z \le 0$ the middle factor is 0, so the whole product is 0 and the weight does not move — no matter how wrong the answer was. A unit that outputs 0 for every input in the training set can never recover. You will see exactly this when you look inside the networks that fail to learn XOR.

5 Partial derivatives and the gradient

With several inputs, a **partial derivative** $\partial f / \partial w_i$ is the slope in the w_i direction with every other variable held fixed. Stack them and you have the **gradient**:

$$\nabla_{\boldsymbol{w}} f = \Big(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_p} \Big).$$

> **The gradient points uphill.** It is the direction of steepest *increase*, and its length says how steep. To decrease a loss you therefore step in $-\nabla$. Every minus sign in every learning rule in this course is that sentence.

Two gradients worth knowing by sight, because they turn a page of algebra into one line:

$$\nabla_{\boldsymbol{w}} \, (\boldsymbol{w} \cdot \boldsymbol{x}) = \boldsymbol{x}, \qquad \nabla_{\boldsymbol{w}} \, \tfrac{1}{2} \|\boldsymbol{w}\|^2 = \boldsymbol{w}.$$

6 Gradient descent, and where the delta rule comes from

The algorithm, in one line, repeated:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \, \nabla_{\boldsymbol{w}} L.$$

Now take a linear neuron $\hat{y} = \boldsymbol{w} \cdot \boldsymbol{x}$ and the squared error $L = \frac{1}{2}(y - \hat{y})^2$, and turn the handle:

$$\frac{\partial L}{\partial \boldsymbol{w}} = (\hat{y} - y) \cdot \frac{\partial \hat{y}}{\partial \boldsymbol{w}} = (\hat{y} - y)\, \boldsymbol{x} \qquad \Longrightarrow \qquad \Delta \boldsymbol{w} = \eta\, (y - \hat{y})\, \boldsymbol{x}.$$

> **That is the delta rule**, and also Rescorla–Wagner, and also the LMS rule in signal processing. Three literatures, three names, one derivation — *move the weights in proportion to the error, along the input.* If you can reproduce these two lines you never have to memorize any of the three.

6.1 The learning rate is the whole difficulty

η too small	loss falls, slowly and smoothly; you run out of patience
η about right	loss falls fast, then flattens
η too large	loss oscillates, or jumps to `nan` within a few steps

Plot the loss on a **log** y-axis (`plt.semilogy`). Progress that looks like a flat line on a linear axis is often a clean straight descent on a log axis, and the difference decides whether you conclude "it is not learning" or "it needs more epochs".

A decaying learning rate, $\eta_t = \eta_0/(1 + t)$, is the classical choice: it shrinks fast enough to settle but slowly enough to still travel any finite distance. That is the Robbins–Monro condition, and it is worth remembering when the fashionable schedulers show up later in the term.

7 Checking a gradient you derived by hand

The single most useful debugging tool in this part of the course. A derivative is a limit of a difference quotient, so approximate it numerically and compare:

```
def numerical_grad(f, w, eps=1e-5):
   g = np.zeros_like(w, dtype=float)
   for i in range(w.size):
      wp, wm = w.astype(float).copy(), w.astype(float).copy()
      wp[i] += eps
      wm[i] -= eps
      g[i] = (f(wp) - f(wm)) / (2 * eps)
   return g

# analytic vs numerical -- these should agree to ~1e-7
np.abs(my_grad(w) - numerical_grad(loss, w)).max()
```

Use the *two-sided* difference shown here, not $(f(w + \epsilon) - f(w))/\epsilon$: it is far more accurate for the same ϵ. It is slow — one pass per parameter — so run it once on a tiny example, confirm the match, then never again.

If the check fails. A factor of exactly 2 means a missing $\frac{1}{2}$ in the loss. A sign flip means you wrote $(y - \hat{y})$ where the derivative gives $(\hat{y} - y)$. Agreement everywhere except at a few entries usually means a ReLU sitting exactly at its kink.

8 Why gradient descent can fail

For a linear neuron with squared error the loss surface is a bowl: one minimum, and descent finds it from anywhere. The moment you add a hidden layer this stops being true, and three things become possible.

- **Local minima** — a valley that is not the deepest valley. Descent stops there because every direction is uphill.
- **Saddle points** — uphill in one direction, downhill in another, with a near-zero gradient in between. Progress stalls without the loss being low.
- **Dead units** — the ReLU case above: a gradient of exactly zero, permanently.

The practical consequence. With a non-convex loss, *the same network trained twice with different random initializations can give different answers, and one of them can simply fail.* This is not a bug to fix. It means a single training run is a single observation, and a claim about an architecture needs several seeds — the same discipline that Recitation 3 applied to t-SNE.

9 Exercises

1. Differentiate $L = \frac{1}{2}(y - wx)^2$ with respect to w, treating x and y as constants. Then write the gradient-descent update.

2. A unit computes $a = \mathrm{ReLU}(\boldsymbol{w} \cdot \boldsymbol{x})$ and currently has $\boldsymbol{w} \cdot \boldsymbol{x} = -3$ while the target is $a = 5$. What is $\partial L / \partial \boldsymbol{w}$, and what happens on the next update?

3. You normalize every input patch to unit length and find that a unit's response now correlates 0.98 with $\cos\theta$ between the patch and the weights, where before normalizing it correlated 0.31. Explain, in one sentence, using $\|\boldsymbol{w}\|\|\boldsymbol{x}\|\cos\theta$.

4. Your analytic gradient is exactly twice your numerical gradient everywhere. What is the bug?

5. A training loss looks like a flat line for 500 epochs. Name two different things this could be, and the one plot that distinguishes them.

6. Two units have preferred orientations of $175°$ and $5°$. A classmate reports their difference as $170°$. What is wrong, and what is the right answer?

Answers

1. $\dfrac{dL}{dw} = -(y - wx)\,x$, so $w \leftarrow w + \eta\,(y - wx)\,x$. The sign flips because we step against the gradient.

2. The ReLU factor $\mathbb{1}[z > 0]$ is 0, so the whole gradient is $\mathbf{0}$ and nothing changes — no matter how large the error. The unit is dead for this input.

3. Before normalizing, the response confounded two causes, $\|\boldsymbol{x}\|$ and $\cos\theta$, and variation in patch intensity dominated. Fixing $\|\boldsymbol{x}\| = 1$ removes the first cause, leaving the response proportional to $\cos\theta$ alone.

4. A missing factor of $\frac{1}{2}$ in the loss: you differentiated $(y - \hat{y})^2$ but your analytic expression is for $\frac{1}{2}(y - \hat{y})^2$, or the reverse.

5. (i) The learning rate is far too small and it is descending imperceptibly; (ii) it is genuinely stuck — dead units, or a saddle. Plot the loss on a log axis: a steady straight descent means (i), a truly flat line means (ii).

6. Orientation is defined modulo $180°$, so the difference is $\min(170, 180 - 170) = 10°$. The two units have nearly the same preference.

Cheat sheet

You want	You write
length of a vector	`np.linalg.norm(x)`
unit-length version	`x / np.linalg.norm(x)`
angle between two vectors	`np.degrees(np.arccos(np.clip(c,-1,1)))`
degrees $\leftrightarrow$ radians	`np.deg2rad` / `np.rad2deg`
orientation difference	`d = abs(a-b) % 180; min(d, 180-d)`
a grating	`np.sin(2*np.pi*f*proj + phase)`
gradient-descent step	`w -= eta * grad`
delta rule	`w += eta * (y - yhat) * x`
loss curve worth reading	`plt.semilogy(losses)`
check a hand-derived gradient	two-sided difference, `eps=1e-5`